

Research Proposal: Investigating Integrations of the Flang Parser with OP2

Sam Armstrong

2026

Abstract

OP2, the Oxford Parallel library for Unstructured mesh solvers, currently relies on Python-based source-to-source translation tools using the `fparser2` library to generate parallelised code for multi-core and many-core architectures from Fortran source code. This approach suffers from robustness, poor maintainability, and performance overhead inherent to interpreted Python. This research investigates the integration of LLVM Flang, a modern C++ compiler frontend supporting the Fortran 2023 standard, as a replacement for `fparser2` within the OP2 translation pipeline. Following the precedent established by OP2-Clang for the C/C++ side of OP2, this work addresses the identified gap for the Fortran side. The project will comprise of three stages - parsing Fortran source into a Flang AST and extracting loop and kernel information; validating kernel correctness with respect to OP2's argument access restrictions; and, as an extension, translating Fortran kernel subroutines to C++ using the Flang AST. Each stage will be evaluated against the existing solution in terms of generated code correctness, translation performance, and robustness of error reporting. The work aims to deliver concrete contributions to the OP2 codebase alongside a comprehensive dissertation report.

Contents

1	Background	3
1.1	Unstructured Mesh Applications	3
1.2	OP2	3
1.2.1	Key Components of the Codebase	4
2	Related Work	4
2.1	Python Parsers for Source-to-Source Translation	4
2.2	Compiler Frontend Parsers	4
2.3	Precedent	5
3	Motivation	5
4	Research Aims and Methodology	6
4.1	Stage 1 - Flang AST Analysis	6
4.1.1	Implementation Details	6
4.1.2	Evaluation Criteria	6
4.2	Stage 2 - Kernel Validation	7
4.2.1	Implementation Details	7
4.2.2	Evaluation Criteria	7
4.3	Stage 3 - Fortran to C++ Translation (Extension)	7
5	Project Management	8
5.1	Timeline	8
5.1.1	Milestones and Deliverables	8
5.2	Risks and Constraints	8
6	Conclusion	9
	References	10

1 Background

1.1 Unstructured Mesh Applications

Mesh solvers are computational frameworks used to simulate physical phenomena on grids (meshes). Structured meshes follow a predictable pattern in their simulation topology, meaning that the mesh can fit logically into a block of contiguous memory. This reduces the need for complex gather/scatter memory operations and also allows for strong spatial locality (where neighbouring cells can be stored next to each other in memory), making these simulations highly performant, particularly on applications that are memory-bound (primarily bottlenecked by memory rather than compute) [1]. In contrast, unstructured meshes have irregular topologies, where each element can have an arbitrary number of neighbours, and use explicit connectivity information to specify how elements are arranged. The distinction is illustrated in Figure 1. Unstructured meshes are essential for handling certain real-world geometries (like Airfoil [2]), where fitting a regular grid cannot create an accurate representation of the physical system; however, it leads to potentially complex data dependencies. These require complex gathers and scatters where the locations of data cannot be determined at compile time, and can also cause race conditions on that data that require additional synchronisation overhead to manage [3]. This makes unstructured mesh applications more difficult to parallelise and run efficiently.

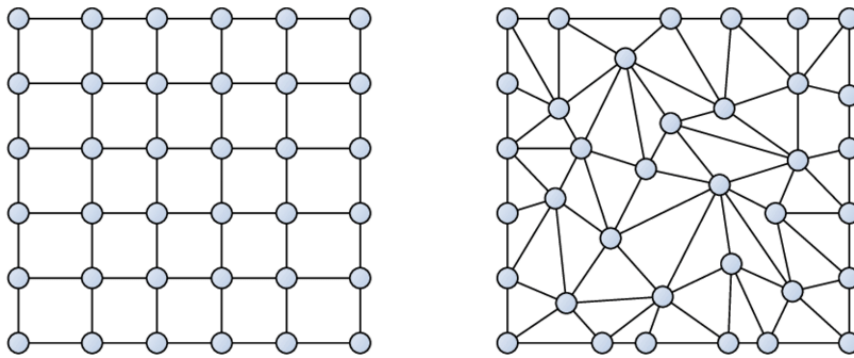


Figure 1: An example of structured (left) and unstructured (right) mesh topologies [4]

1.2 OP2

Oxford Parallel library for Unstructured mesh solvers, shortened to OP2, is a high-level embedded domain specific language for enabling unstructured mesh applications to be automatically parallelised and executed on multi-core and many-core architectures [5]. OP2 allows users to write a single application using the high-level API in Fortran, C or C++ to describe the mesh and the operations to be performed on it. This describes the application from a purely mathematical and logical perspective, rather than providing precise details for how the application should be parallelised. Before compiling the program, the user uses the Python-based source-to-source translation scripts to automatically convert the generic application they’ve written into low-level code for each target backend that’s supported on their system, generating versions of the code which can be run efficiently on any specific architecture. This generated code can then be compiled, which links the OP2 runtime functions, creating an optimised executable which can be run on any of the target hardware. Various applications using OP2 have been published [6, 7, 8] that demonstrate strong performance on a wide range of architectures including multi-core CPUs, GPUs, and clusters of CPUs.

1.2.1 Key Components of the Codebase

`/op2` - Contains the C/C++ OP2 runtime libraries and Fortran bindings.

`/translator` - Contains the code for the original Python-based translators that convert the Fortran, C or C++ source code using OP2's API into parallel executable versions for different backends, such as OpenMP and CUDA. This is accomplished by parsing the source code using regular expressions and string manipulation to identify constructs like `op_par_loop` and `op_init`, and doesn't build an abstract syntax tree of the program, meaning they are somewhat error prone and lack robustness.

`/translator-v2` - Contains an updated implementation of the translators which uses a more modular software design. It uses LibTooling [9] (for C/C++) and `fparser2` [10] (for Fortran) for parsing the source code and Jinja2 for templating the generated code. These make it generally better than the legacy translators, due to the AST parsing, superior code generation and improved maintainability, but comes at the cost of more dependencies, and requiring Python 3.8 or above.

2 Related Work

2.1 Python Parsers for Source-to-Source Translation

Python as a whole has been used in many related domain specific languages and active library frameworks for code generation targeting specific hardware architectures, like OPS [11], Fire-drake [12], OpenSBLI [13] and DeVito [14]. The main issue with using Python in these contexts is that tools written in Python significantly lack the robustness of compiler frameworks such as Clang or GNU [15], as they have very limited syntactic or semantic checking, leading to poor error and warning reporting. This has downstream effects on the maintainability and extensibility of the tools that utilise them, as the code becomes complicated when adding new features or optimisations, making it difficult to debug.

The Fortran parser used in the existing OP2 translators is `fparser2` [10], which is used to parse the source code into an AST, allowing the program to be easily inspected and manipulated. There are several benefits to using it in the context of OP2 - it can be easily installed via pip as a Python dependency; the AST nodes produced are Python objects, making them easy to interact with; and it can parse source code without requiring the full semantic context of the program, meaning files can be parsed independently of each other.

However, this lack of semantic context is also a significant limitation, as with the use of Python in general, due to the poor maintainability and extensibility, which has ramifications for OP2 in its current implementation. Another key issue with the use of `fparser2` is performance - as a parser that operates purely in Python, it is significantly slower than a compiled C++ parser, which can make the OP2 source-to-source translation process take a long time to complete, particularly on large programs. There is also an issue of the support and maintenance. `fparser2` only officially supports Fortran 2003 and 'some' of 2008 [10], meaning it lags behind full compiler frontends in supporting the latest standard features.

2.2 Compiler Frontend Parsers

A compiler frontend is the part of a compiler that parses the raw source code into an AST, then converts the AST into an intermediate representation, such as LLVM IR, which can be optimised and compiled to machine code by the rest of the compiler. Such compiler tooling has been explored in the context of code generation tasks [16, 17, 18] such as source-to-source translation, and was applied to the C/C++ side of OP2 in OP2-Clang [15].

One of the modern compiler frontends for Fortran is LLVM Flang [19] (Flang) - the frontend for the LLVM compiler infrastructure [20]. Several previous works have explored the use of Flang for the parallelisation of annotated Fortran code [21, 22, 23], demonstrating the use of its parser for translation applications in high performance computing.

Flang therefore offers a potential alternative to fparser2 that could be used for the Fortran side of OP2. There are several advantages to doing this, primarily due to it being a full compiler frontend for Fortran and it being written in C++. As a full compiler frontend, Flang supports up to the latest Fortran 2023 standard [19], and has better support across the board than fparser2, meaning it will be significantly more reliable for use in OP2. In addition, LLVM Flang being written in C++ means it's compiled into a highly optimised executable, which should make it significantly faster than fparser2, which uses relatively slow interpreted Python.

The potential downsides of using Flang are that it's a heavy dependency to add to the OP2 codebase; it is a relatively new project, which could result in issues with its stability; and it could be difficult to integrate into the existing OP2 codebase. We'll explore the trade-offs of these potential upsides and downsides of using Flang compared to fparser2 in the context of OP2 in this research, alongside its integrations and use cases.

2.3 Precedent

The precedent for this work has already been laid out by OP2-Clang [15], which addresses a similar problem to ours from the C++ side of OP2 - replacing the use of the Python-based source-to-source translator with one based on Clang LibTooling. This implementation uses LibTooling's `ASTMatchers` and `Replacements` to modify a skeleton to generate the necessary parallel code, then uses multiple levels of refactoring using LibTooling's `RefactoringTool` to apply specific optimisations in a 'flexible, maintainable and extensible manner'. Two main issues were identified with past works in using source-to-source translation tools to help with writing programs for new hardware [24, 25] that motivated this research - poor extensibility for generating new parallelisations, and the underlying source-to-source translation tool using unreliable or unmaintained software.

This work is reflected in the `/translator-v2` directory of the OP2 codebase, but differs slightly from our own in that this second version of the translator already uses an AST-based approach in Python for translating the Fortran source code, whereas OP2-Clang was comparing directly to the first version of the translator, which did not use an AST-based approach. However, the overall objective of the research is essentially the same - to replace the use of a Python-based source-to-source translator with one based on a more robust parser.

The evaluation of OP2-Clang was done by comparing the performance of the generated code to the performance of the code generated by OP2's current Python-based source-to-source translator. The generated code showed near-identical performance to the existing solution, indicating that the code generated by OP2-Clang is likely identical, though this was not rigorously tested and verified.

3 Motivation

Given the related work discussed, there is a clear gap in the literature for integrating Flang into OP2. Primarily, the use of the Flang LLVM frontend will improve the robustness and maintainability of the OP2 codebase, as it is a full compiler frontend for Fortran, meaning it has far better support than fparser2. It should also make the translation process more performant, being a C++ based parser and being compiled with aggressive optimisations. The code generated by using Flang should also, theoretically, be identical to the code generated by

the current source-to-source translator, meaning there is little trade-off other than introducing this new dependency.

The integration of Flang into OP2 was also identified as a point of future work in the OP2-Clang paper - ‘this work can be extended for OP2’s Fortran API, if a similar infrastructure is developed around the upcoming Flang front end’ [15].

4 Research Aims and Methodology

There are three stages that will make up this research project, each aimed at addressing one of the following research questions:

- *Stage 1:* Can LLVM Flang be used to parse OP2 Fortran source code and extract loop and kernel information with equivalent correctness to the existing translator, and with measurably better performance?
- *Stage 2:* Can an AST analysis with Flang reliably validate OP2 kernel argument access restrictions, producing error reporting that is at least as informative as the current Python implementation?
- *Stage 3:* Is it feasible to use Flang to translate Fortran kernel subroutines to C++ as a replacement for the current translation, while maintaining or improving performance?

4.1 Stage 1 - Flang AST Analysis

In this initial stage, we’ll be aiming to parse the input Fortran source code into an AST using Flang, and extract all the necessary information. This will include the argument information for each `op_par_loop`, and the source code for each of the kernels, including all functions and subroutines that are called.

4.1.1 Implementation Details

The first thing we’ll need as output is the argument information for each `op_par_loop`, which should be fairly straightforward to extract by walking the Flang AST. The second thing is the source code for each of the kernels, including all functions and subroutines that are called. This will require resolving the reference to the kernel subroutine, then recursively finding the tree of dependencies. We should be able to use AST unparsing to then convert each of these back into source code [26], or if this proves infeasible, we’ll need to use the source line and character information to extract excerpts from the original source code.

4.1.2 Evaluation Criteria

To verify the correctness of the extracted information, the outputs will be compared directly against the output of the existing translator on a suite of OP2 applications, including Airfoil and the other OP2 benchmarks. This will involve a structured comparison of the extracted `op_par_loop` argument lists and kernel dependency trees, checking for equivalence. Once this is confirmed, we’ll use performance benchmarking to validate the correctness of the generated code. We’ll also use benchmarking of the source-to-source translator itself to evaluate the performance with Flang compared to the existing solution, as we’d expect Flang to be significantly faster than `fparser2`.

4.2 Stage 2 - Kernel Validation

This followup stage will focus on validating that the kernels only perform operations permitted by the specified argument access types and other restrictions, using loop metadata and the extracted kernels provided by Flang - functionality that is currently implemented using the `fparser2` AST.

4.2.1 Implementation Details

This implementation will require tracking arguments through child subroutine calls, which will be achieved by walking the Flang parse tree recursively, maintaining a table that maps parameters to their access types. This will then be used to check the kernel's validity with respect to restrictions like:

- *Access Type Validation* - ensuring that the kernels only use arguments in the valid ways, such as arguments marked as `OP_READ` never being written to, or arguments marked as `OP_INC` are only updated via increment operations and not arbitrary assignments.
- *Global Constant Protection* - checking that global constants (`op_decl_const`) are not modified within the kernel.
- *Argument Consistency* - validating the number of arguments passed to `op_par_loop` matches the number of parameters defined in the kernel subroutine.
- *Dependency Analysis* - checking that all dependencies represent valid references.

4.2.2 Evaluation Criteria

Evaluation of this stage will consist of creating a test suite of kernels that are known to be valid and invalid, using this to verify that the kernel validation is working correctly, and in the same manner as the current source-to-source translator. We'll also use this test suite to ensure that error messages and warnings are still reported correctly and are informative, which is a potential risk of using a different parser. The test suite will be constructed to provide coverage across each of the four validation categories, with at least one valid and one invalid kernel for each. Similarly to stage 1, we'll also use performance benchmarking to compare the performance of the new and existing solutions with regard to kernel validation.

4.3 Stage 3 - Fortran to C++ Translation (Extension)

OP2 currently translates Fortran kernel subroutines into C++ for greater compiler compatibility and device targeting using the `fparser2` AST. As in the other sections, the aim will be to replace the use of `fparser2` with Flang for better robustness and performance, maintaining the same process of parsing the AST to extract kernel metadata and parameter lists, mapping intrinsic types to their C++ equivalents and evaluating variable mutability, and using a dispatch table to produce the C++ code. The evaluation will compare the translated C++ output against the code generated by the existing solution on the same kernel suite used in stage 2, checking for both syntactic equivalence and runtime correctness by compiling and executing the translated kernels against known outputs. We're considering this stage as a research extension, so its completion will be dependent on the success and timely completion of the previous stages.

5 Project Management

5.1 Timeline

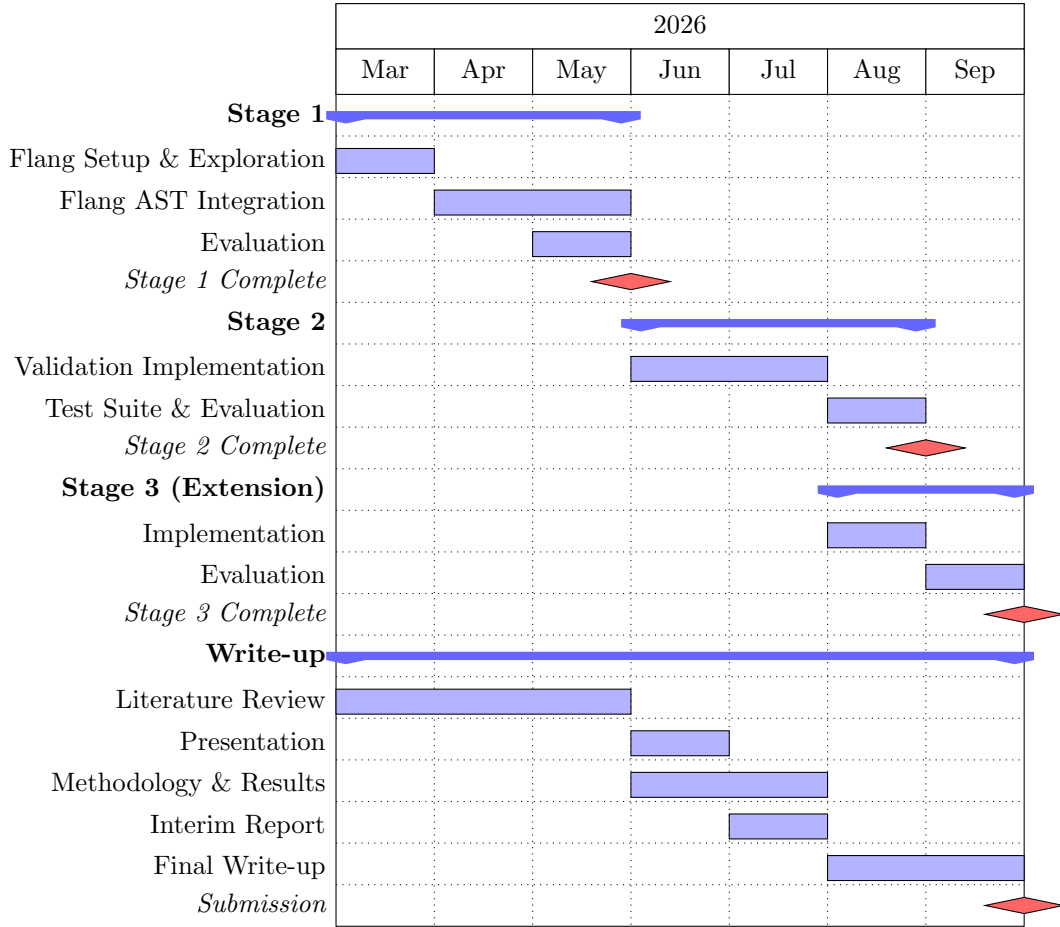


Figure 2: Gantt chart of the project timeline

5.1.1 Milestones and Deliverables

As shown in Figure 2, the project is expected to take just over 6 months to complete, to hit our final submission deadline of September 8th 2026, with the key checkpoints being the completion of stages 1 and 2, and stage 3 being a research extension that we're hoping to complete if time allows. For the write-up process, the key milestones will be the completion of the literature review and the methodology and results gathering; with the presentation, interim report, and final write-up being the hard deadlines for write-up deliverables.

In terms of code deliverables, the completion of each stage will involve a pull request to the OP2 codebase containing the implementation of the stage, the results it delivered, and the relevant tests to verify its correctness. For a stage to be considered complete, it must pass all OP2 test and example cases (such as Airfoil), and produce generated code that is within 2% of the performance of the existing solution - the same success criteria used in OP2-Clang.

5.2 Risks and Constraints

There are a few risks and limitations of this research that are worth considering before it commences. One key risk is the difficulty of integrating Flang into the OP2 codebase, which could be problematic for delivering the key milestones on time. This could be particularly

relevant for stage 2, which may be quite involved as we'll need to track arguments through child subroutine calls to check the validity of the kernels. To mitigate this risk, stage 3 is being considered as an optional extension, so it could potentially be dropped if time ends up being a significant constraint. We'll also begin the project with an exploration phase in which Flang will be tested against simple OP2 Fortran programs before any integration work begins. If argument tracking through nested subroutine calls in stage 2 proves intractable within the available time, the scope will be reduced to single-level kernel validation, which would still provide research value that can be expanded upon in future work.

Another risk is that LLVM Flang is a relatively new project, meaning there may be issues with its stability and reliability that could affect its integration into OP2. As we know this is a risk, this has been taken into account when creating the project timeline, allowing for time to create workarounds or submit patches to the Flang project if necessary. If issues with this persist, we may retain some `fparser2` use to cover the problematic areas, meaning the integration would only be partial, but it would not block progress on the project.

Finally, there is a potential risk that using a different parser could cause error messages and warnings to be reported incorrectly or lose their informativeness compared to the current source-to-source translator. To address this, a test suite of deliberately invalid OP2 programs will be created that will be used for regression testing throughout the course of the project.

6 Conclusion

This research project aims to address the gap in the literature and proposed future work of integrating Flang into OP2, to provide more robust and performant Fortran source-to-source translators in OP2. It will offer material deliverables in the form of code changes to the OP2 codebase, and will showcase evaluation of the performance of both the translation process and the generated code, as well as the correctness and robustness of the pipeline compared with the existing Python implementation in OP2, through a comprehensive dissertation report.

At this early stage, there are several questions that remain open about the course of the research. For example, it is unknown whether the flexibility of an interpreted Python parser, which can operate without full semantic context, is an advantage that a compiled compiler frontend cannot easily replicate, and this trade-off will be a key point of evaluation. Likewise, it is not yet certain that Flang's AST unparsing capabilities are practical for reconstructing kernel source code, potentially requiring fallback strategies to be used.

Beyond OP2, this research will help determine whether compiler tooling is a viable and practical replacement for Python-based parsing for translation tasks across the high performance computing domain and inform future design choices for other DSLs, which face similar trade-offs to the ones we're considering.

References

- [1] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [2] M. Giles, Devendra Ghate, and Mihai Duta. Using automatic differentiation for adjoint cfd code development. *Computational Fluid Dynamics Journal*, 16, 01 2008.
- [3] Carlo Bertolli, Adam Betts, Gihan R. Mudalige, Mike B. Giles, and Paul H.J. Kelly. Design and performance of the OP2 library for unstructured mesh applications. In *Euro-Par 2012: Parallel Processing Workshops*, volume 7640 of *Lecture Notes in Computer Science*, pages 191–200. Springer, 2013.
- [4] Mohamed Aissa. *GPU-accelerated CFD Simulations for Turbomachinery Design Optimization*. PhD thesis, 10 2017.
- [5] G.R. Mudalige, M.B. Giles, I. Reguly, C. Bertolli, and P.H.J. Kelly. Op2: An active library framework for solving unstructured mesh-based applications on multi-core and many-core architectures. In *2012 Innovative Parallel Computing (InPar)*, pages 1–12, 2012.
- [6] I. Z. Reguly, D. Gopinathan, J. H. Beck, M. B. Giles, S. Guillas, and F. Dias. The Volna-OP2 tsunami code (version 1.0). *Geoscientific Model Development Discussions*, 2018.
- [7] N. Jacobsen. LLVM supported source-to-source translation: Translation from annotated C/C++ to CUDA C/C++. Master’s thesis, University of Oslo, 2016.
- [8] M.B. Giles, G.R. Mudalige, Z. Sharif, G. Markall, and P.H.J. Kelly. Performance analysis and optimization of the op2 framework on many-core architectures. *The Computer Journal*, 55(2):168–180, 2012.
- [9] The LLVM Project. “libtooling”, 2026.
- [10] Andrew Porter, Rupert Ford, Balthasar Reuter, Joerg Henrichs, and Pearu Peterson. fparser2: A fortran parser, 2021. <https://fparser.readthedocs.io/en/latest/fparser2.html>.
- [11] I. Z. Reguly, G. R. Mudalige, M. B. Giles, D. Curran, and S. McIntosh-Smith. The OPS domain specific abstraction for multi-block structured grid computations. In *Proceedings of the 2014 Fourth International Workshop on Domain-Specific Languages and High-Level Frameworks for High Performance Computing, WOLFHPC ’14*, pages 58–67. IEEE Computer Society, 2014.
- [12] Florian Rathgeber, David A. Ham, Lawrence Mitchell, Michael Lange, Fabio Luporini, Andrew T. T. Mcrae, Gheorghe-Teodor Bercia, Graham R. Markall, and Paul H. J. Kelly. Firedrake: Automating the finite element method by composing abstractions. *ACM Transactions on Mathematical Software*, 43:24:1–24:27, December 2016.
- [13] Christian T. Jacobs, Satya P. Jammy, and Neil D. Sandham. OpenSBLI: A framework for the automated derivation and parallel execution of finite difference solvers on a range of computer architectures. *Journal of Computational Science*, 18:12–23, 2017.
- [14] Michael Lange, Navjot Kukreja, Mathias Louboutin, Fabio Luporini, Felipe Vieira, Vincenzo Pandolfo, Paulius Velesko, Paulius Kazakas, and Gerard Gorman. Devito: Towards a generic finite difference DSL using symbolic Python. In *Proceedings of the 6th Workshop on Python for High-Performance and Scientific Computing, PyHPC ’16*, pages 67–75, Piscataway, NJ, USA, 2016. IEEE Press.

- [15] G.D. Balogh, G.R. Mudalige, I.Z. Reguly, S.F. Antao, and C. Bertolli. Op2-clang: A source-to-source translator using clang/llvm libtooling. In *2018 IEEE/ACM 5th Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*, pages 59–70, 2018.
- [16] Eli Bendersky. Modern source-to-source transformation with Clang and LibTooling, 2014. <https://eli.thegreenplace.net/2014/07/29/ast-matchers-and-clang-refactoring-tools>.
- [17] Jingyue Wu, Artem Belevich, Eli Bendersky, Mark Heffernan, Chris Leary, Jacques Pienaar, Bjarke Roune, Rob Springer, Xuettian Weng, and Robert Hundt. gpucc: An open-source GPGPU compiler. In *Proceedings of the 2016 International Symposium on Code Generation and Optimization*, CGO ’16, pages 105–116. ACM, 2016.
- [18] Matteo Marangoni and Thomas Wischgoll. ToGPU: Automatic source transformation from C++ to CUDA using Clang/LLVM. *Electronic Imaging*, 2016(1):1–9, 2016.
- [19] The LLVM Project. “llvm flang”, 2026. <https://flang.llvm.org/docs/FortranStandardsSupport.html>.
- [20] C. Lattner and V. Adve. Llvm: a compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, pages 75–86, 2004.
- [21] Güray Özen, Simone Atzeni, Michael Wolfe, Annemarie Southwell, and Gary Klimowicz. Openmp gpu offload in flang and llvm. In *2018 IEEE/ACM 5th Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*, pages 1–9, 2018.
- [22] Gabriel Rodriguez-Canal, David Katz, and Nick Brown. An mlir pipeline for offloading fortran to fpgas via openmp. In *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, page 1096–1103. ACM, November 2025.
- [23] Eric Wright, Johannes Doerfert, Shilei Tian, Barbara Chapman, and Sunita Chandrasekaran. Implementing openmp’s simd directive in llvm’s gpu runtime. In *Proceedings of the 52nd International Conference on Parallel Processing, ICPP ’23*, page 173–182, New York, NY, USA, 2023. Association for Computing Machinery.
- [24] Olaf Krzikalla, Konrad Feldhoff, Ralph Müller-Pfefferkorn, and Wolfgang E. Nagel. Scout: A source-to-source transformator for SIMD-optimizations. In *Proceedings of the 2011 International Conference on Parallel Processing – Volume 2, Euro-Par’11*, pages 137–145, Berlin, Heidelberg, 2012. Springer-Verlag.
- [25] Seyong Lee, Seung-Jai Min, and Rudolf Eigenmann. OpenMP to GPGPU: A compiler framework for automatic translation and optimization. *ACM SIGPLAN Notices*, 44(4):101–110, 2009.
- [26] The LLVM Project. Flang Parsing Documentation, 2026. <https://github.com/llvm/llvm-project/blob/main/flang/docs/Parsing.md>.